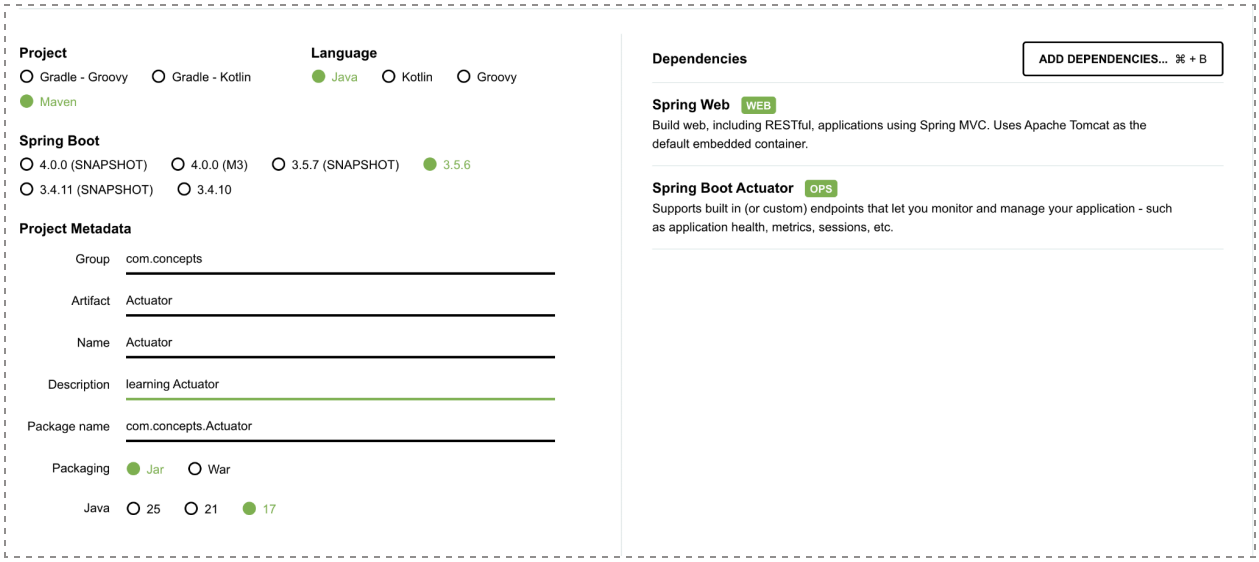
Actuator:

Provides production-ready endpoints to monitor and manage the Spring boot application.

Project setup:

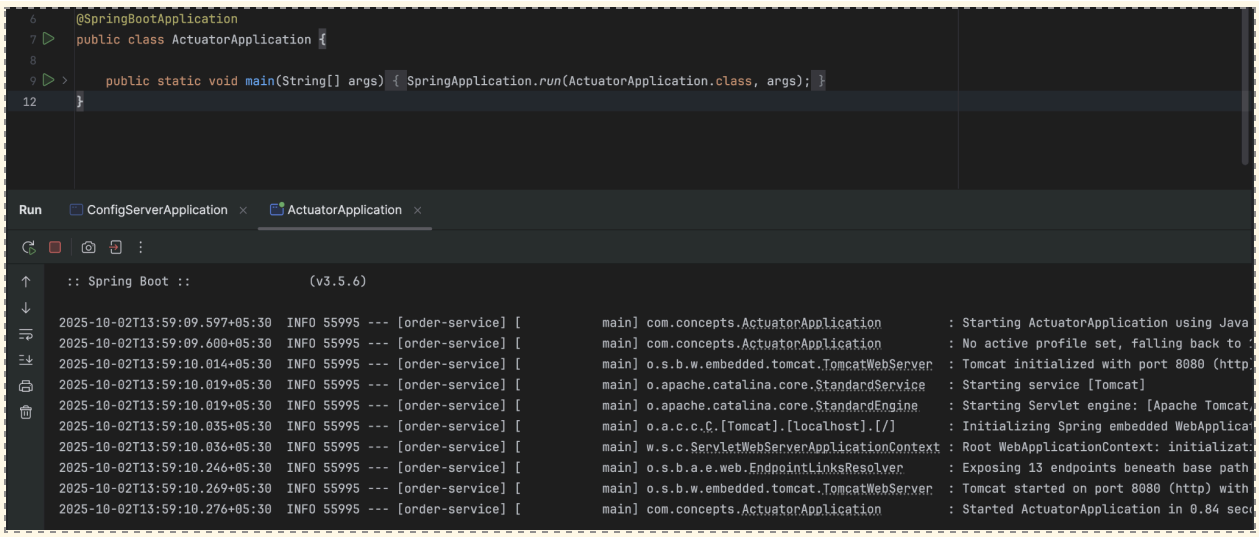


application.properties

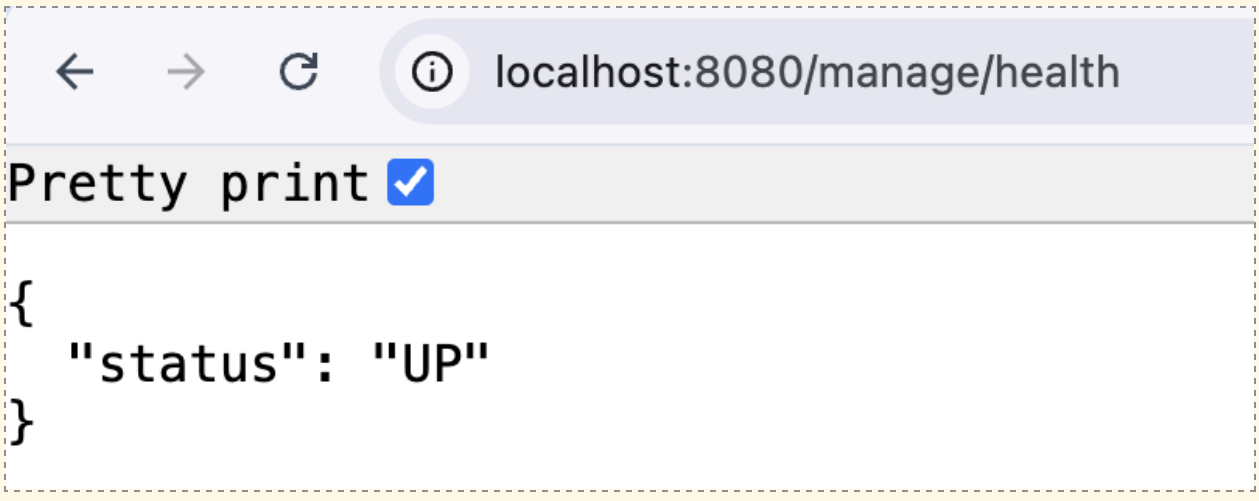
application.properties

```
spring.application.name=order-service

# by-default path is /actuator
management.endpoints.web.base-path=/manage

# Expose all actuator endpoints, by-default '/actuator/health' & '/actuator/info' endpoint is exposed
# '*' expose all the endpoints
# use comma separated like: health, info, metrics, loggers etc. to expose selected endpoints
management.endpoints.web.exposure.include=*


```

Let's start the server and hit the Actuator endpoints:

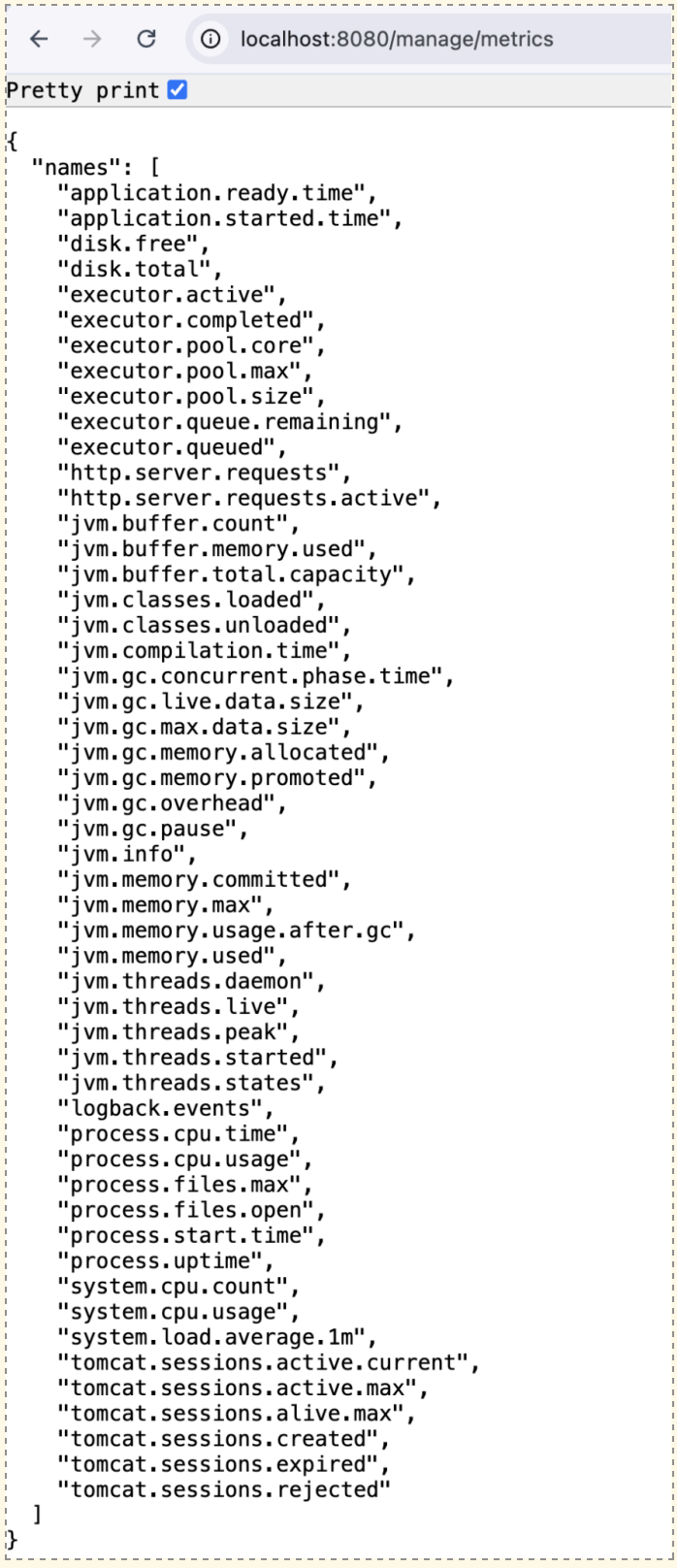


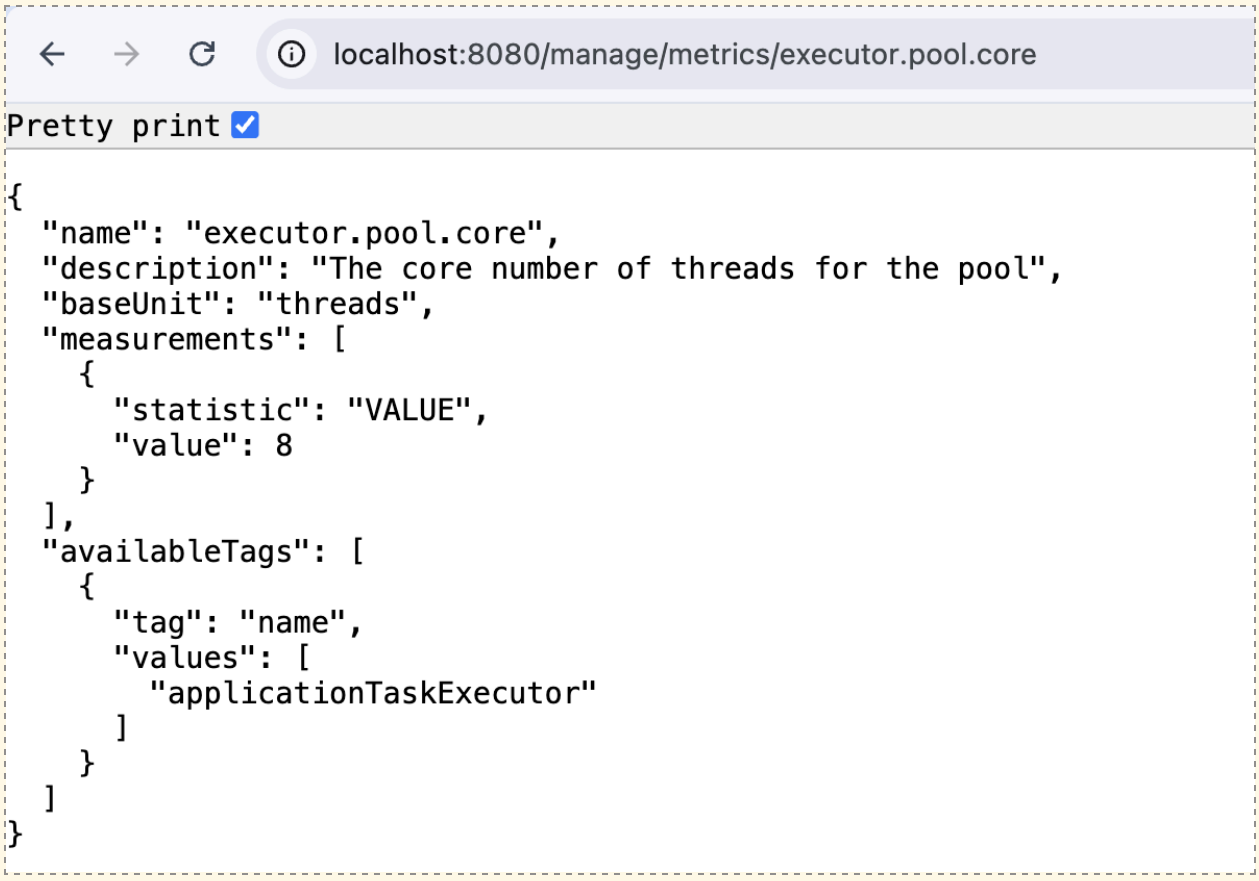
Health API : '/health'



Metrics API : '/metrics'

/metrics' gives list of all metrics endpoint


localhost:8080/manage/metrics
Pretty print
{
"names": [
"application.ready.time",
"application.started.time",
"disk.free",
"disk.total",
"executor.active",
"executor.completed",
"executor.pool.core",
"executor.pool.max",
"executor.pool.size",
"executor.queue.remaining",
"executor.queued",
"http.server.requests",
"http.server.requests.active",
"jvm.buffer.count",
"jvm.buffer.memory.used",
"jvm.buffer.total.capacity",
"jvm.classes.loaded",
"jvm.classes.unloaded",
"jvm.compilation.time",
"jvm.gc.concurrent.phase.time",
"jvm.gc.live.data.size",
"jvm.gc.max.data.size",
"jvm.gc.memory.allocated",
"jvm.gc.memory.promoted",
"jvm.gc.overhead",
"jvm.gc.pause",
"jvm.info",
"jvm.memory.committed",
"jvm.memory.max",
"jvm.memory.usage.after.gc",
"jvm.memory.used",
"jvm.threads.daemon",
"jvm.threads.live",
"jvm.threads.peak",
"jvm.threads.started",
"jvm.threads.states",
"logback.events",
"process.cpu.time",
"process.cpu.usage",
"process.files.max",
"process.files.open",
"process.start.time",
"process.uptime",
"system.cpu.count",
"system.cpu.usage",
"system.load.average.1m",
"tomcat.sessions.active.current",
"tomcat.sessions.active.max",
"tomcat.sessions.alive.max",
"tomcat.sessions.created",
"tomcat.sessions.expired",
"tomcat.sessions.rejected"
]
}

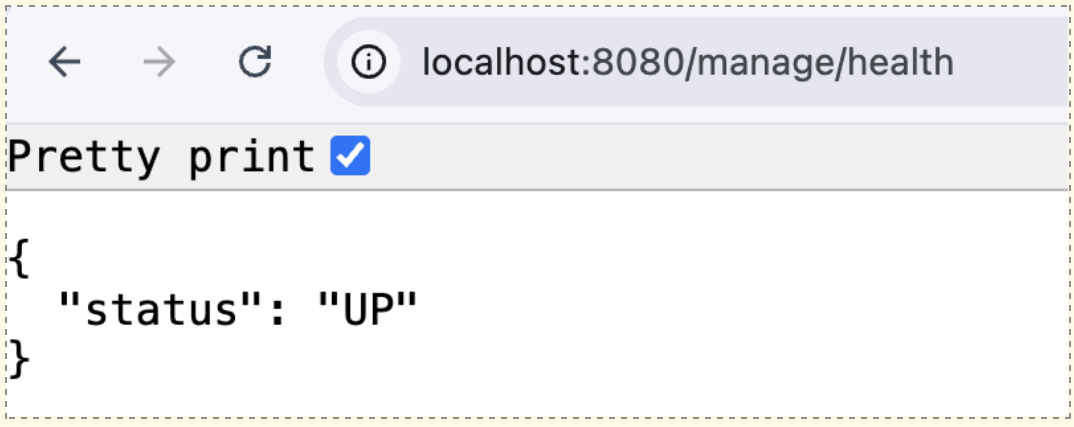


Hitting specific metrics endpoint

Lets first see some useful in-built Actuator endpoints:

GET: /health

Provides health status of our application: UP, DOWN, OUT_OF_SERVICE, UNKNOWN



/health endpoint can be extended to add additional checks like DB health, Cache status etc. beyond just application server status

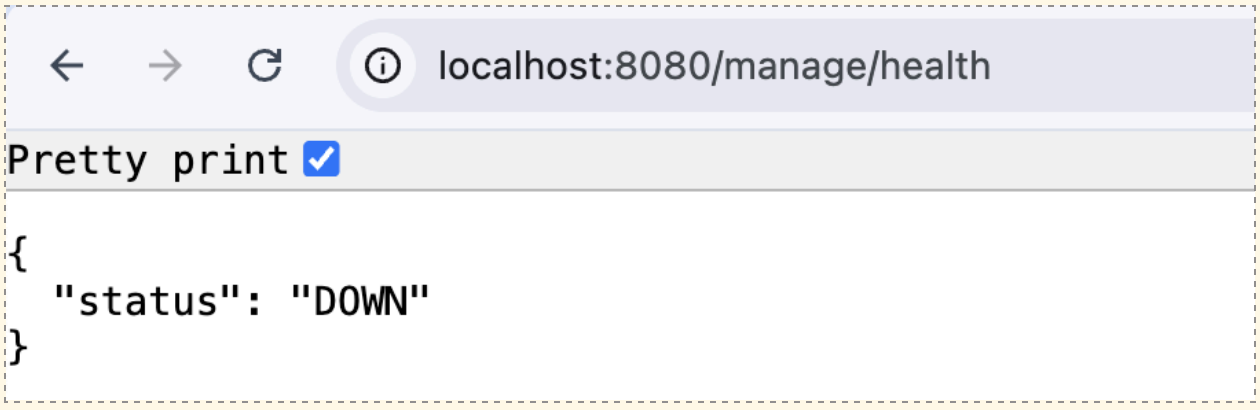
Adding DB health check:

```java
@Component
public class DatabaseHealthIndicator implements HealthIndicator {
    @Override
    public Health health() {
        boolean isDBUp = checkDBConnection();
        return isDBUp ? Health.up().withDetail("DB", "Available").build()
                : Health.down().withDetail("DB", "Not-Available").build();
    }

    private boolean checkDBConnection() {
        //check  DB is up or not
        return true;
    }
}
```

Adding Cache health check:

```java
@Component
public class CacheHealthIndicator implements HealthIndicator {
    @Override
    public Health health() {
        boolean isCacheUp = checkCacheStatus();
        return isCacheUp ? Health.up().withDetail("Cache", "Available").build()
                : Health.down().withDetail("Cache", "Not-Available").build();
    }

    private boolean checkCacheStatus() {
        //check  cache status
        return false;
    }
}
```

By-default, its shows only the overall status of the application



But if we want more details, we need to add:

application.properties

```
spring.application.name=order-service

# by-default path is /actuator
management.endpoints.web.base-path=/manage

# Expose all actuator endpoints, by-default '/actuator/health' & '/actuator/info' endpoint is exposed
# '*' expose all the endpoints
# use comma separated like: health, info, metrics, loggers etc. to expose selected endpoints
management.endpoints.web.exposure.include=*

#to view the details in health endpoint
management.endpoint.health.show-details=always

```

GET: /metrics
and
/metrics/{metric name}

There are so many metrics, just showing some important ones:

	Metric name	What it represents	Example
JVM Memory Metrics			
	jvm.memory.used	Memory currently in use by JVM	{ "statistic": "VALUE", "value": 99478616 }

	jvm.memory.max	Max memory JVM can use	{ "statistic": "VALUE", "value": 10989076477 }
Garbage Collection Metrics			

	jvm.gc.pause	Time spent in GC.
		
		COUNT: total no. of GC events occurred.
		
		TOTAL_TIME: total time spent in GC (usually seconds)
		
		MAX: longest single GC pause observed
			[
			{
			"statistic": "COUNT",
			"value": 12
			},
			{
			"statistic": "TOTAL_TIME",
			"value": 2.305
			},
			{
			"statistic": "MAX",
			"value": 0.9
			}
			]

Threads			

	jvm.threads.live	Number of live threads	{ "statistic": "VALUE", "value": 22 }
	jvm.threads.peak	Peak live thread count, since JVM started	{ "statistic": "VALUE", "value": 50 }
System Metrics			
	system.cpu.usage	CPU used by JVM (range 0.0 - 1.0)	Value: 0.10 -> 10%
HTTP Server / Requests			

	http.server.requests	COUNT: Total no of HTTP requests received TOTAL_TIME: total time spent handling all requests (usually in seconds) MAX: longest time taken to handle a single request	"measurements": [{ "statistic": "COUNT", "value": 152 }, { "statistic": "TOTAL_TIME", "value": 23.45 }, { "statistic": "MAX", "value": 0.89 }], "availableTags": [{ "tag": "method", "values": ["GET", "POST"] },

			{ "tag": "status", "values": ["200", "404", "500"] }] }
Database / JDBC Metric			
	jdbc.connections.active	Connections currently in use	VALUE: 3
	jdbc.connections.idle	Idle connections in pool	VALUE: 7
	jdbc.connections.max	Max connections allowed	VALUE: 10

GET: /threaddump

- Helps to diagnose deadlock or thread leaks.

- **Which threads are active, blocked or waiting.**
- **Stack trace for each thread (what code it is executing)**
- **Thread name, id, priority etc.**


```
[
 {
  "threadName": "main",
  "threadId": 1,
  "blockedTime": -1,
  "blockedCount": 0,
  "waitedTime": -1,
  "waitedCount": 0,
  "threadState": "RUNNABLE",
  "stackTrace": [
   "com.concepts.MyService.methodName(MyService.java:142)",
   "com.concepts.ActuatorApp.main(ActuatorApp.java:10)"
  ]
 },
 {
  "threadName": "thread2",
  "threadId": 2,
  "blockedTime": -2,
  "blockedCount": 0,
  "waitedTime": -1,
  "waitedCount": 0,
  "threadState": "WAITING",
  "stackTrace": [
   "com.concepts.ClassName.methodName(ClassName.java:12)",
   "com.concepts.ActuatorApp.main(ActuatorApp.java:10)"
  ]
 }
]
```

What all other API's available:
Official Documentation:
https://docs.spring.io/spring-boot/reference/actuator/endpoints.html#actuator.endpoints

/heapdump	GET	Downloads JVM heap as .hprof file
/mappings	GET	Lists all Spring MVC request mappings
/beans	GET	Displays a complete list of all the Spring beans in your application
/configprops	GET	Lists all @ConfigurationProperties beans
/loggers	GET	Lists all loggers and their current levels
/shutdown	POST	Lets the application be gracefully shutdown
/env	GET	Shows environment properties
/actuator/env/{property}	GET	Shows a specific environment property

By default, access to all endpoints except for "/shutdown" and "/heapdump" is unrestricted

Both these endpoints are very critical:

- **/shutdown can stop your application**
- **/heapdump can expose sensitive information about your application like tokens, password etc.**

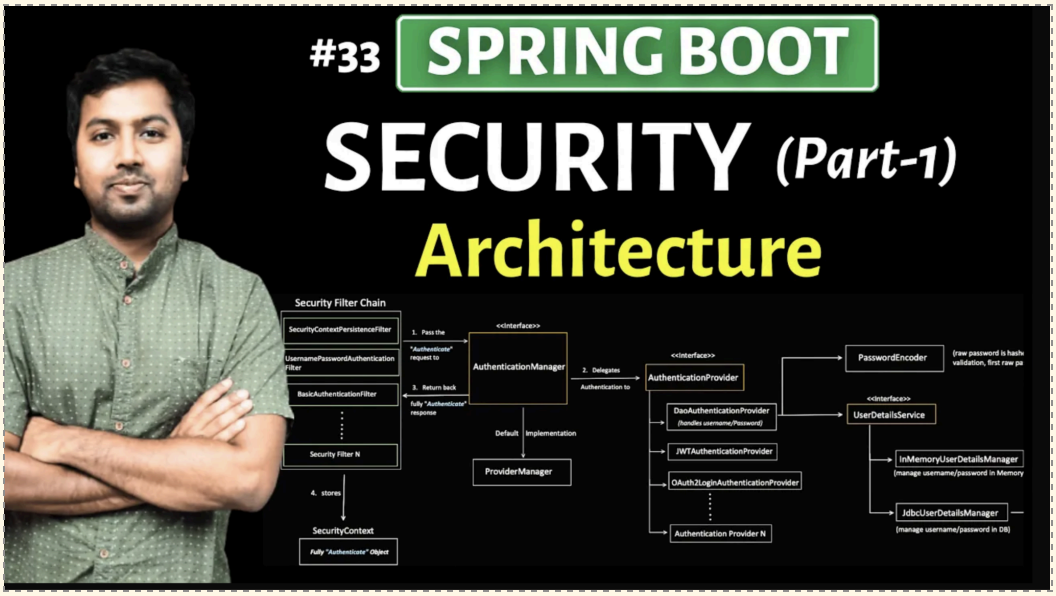
  So, by default they are restricted and specifically want us to unrestricted it (assuming, we have accepted the risk involved)

application.properties

```
# by-default path is /actuator
management.endpoints.web.base-path=/manage

# Expose all actuator endpoints, by-default '/actuator/health' & '/actuator/info' endpoint is exposed
# '*' expose all the endpoints
# use comma separated like: health, info, metrics, loggers etc. to expose selected endpoints
management.endpoints.web.exposure.include=*

#to view the details in health endpoint
management.endpoint.health.show-details=always

# Make it require authentication
management.endpoint.heapdump.access=unrestricted
```

Security

We have already covered Spring Security in depth (total 9 parts), so kindly check it out, if there is any doubt with that:



pom.xml

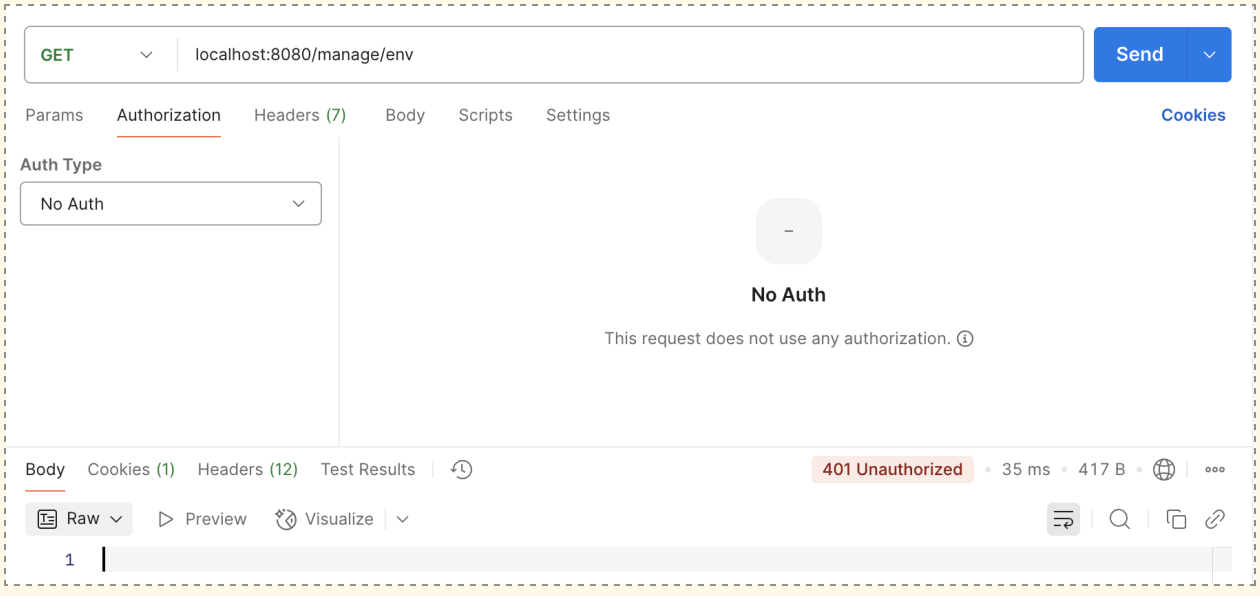
```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

```java
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception{

        http.authorizeHttpRequests(auth -> auth
                        .requestMatchers( ...patterns: "/manage/health", "/manage/info").permitAll() // public
                        .anyRequest().authenticated() // everything else requires login
                )
                .httpBasic(Customizer.withDefaults()); // basic auth for simplicity
        return http.build();

    }
}
```

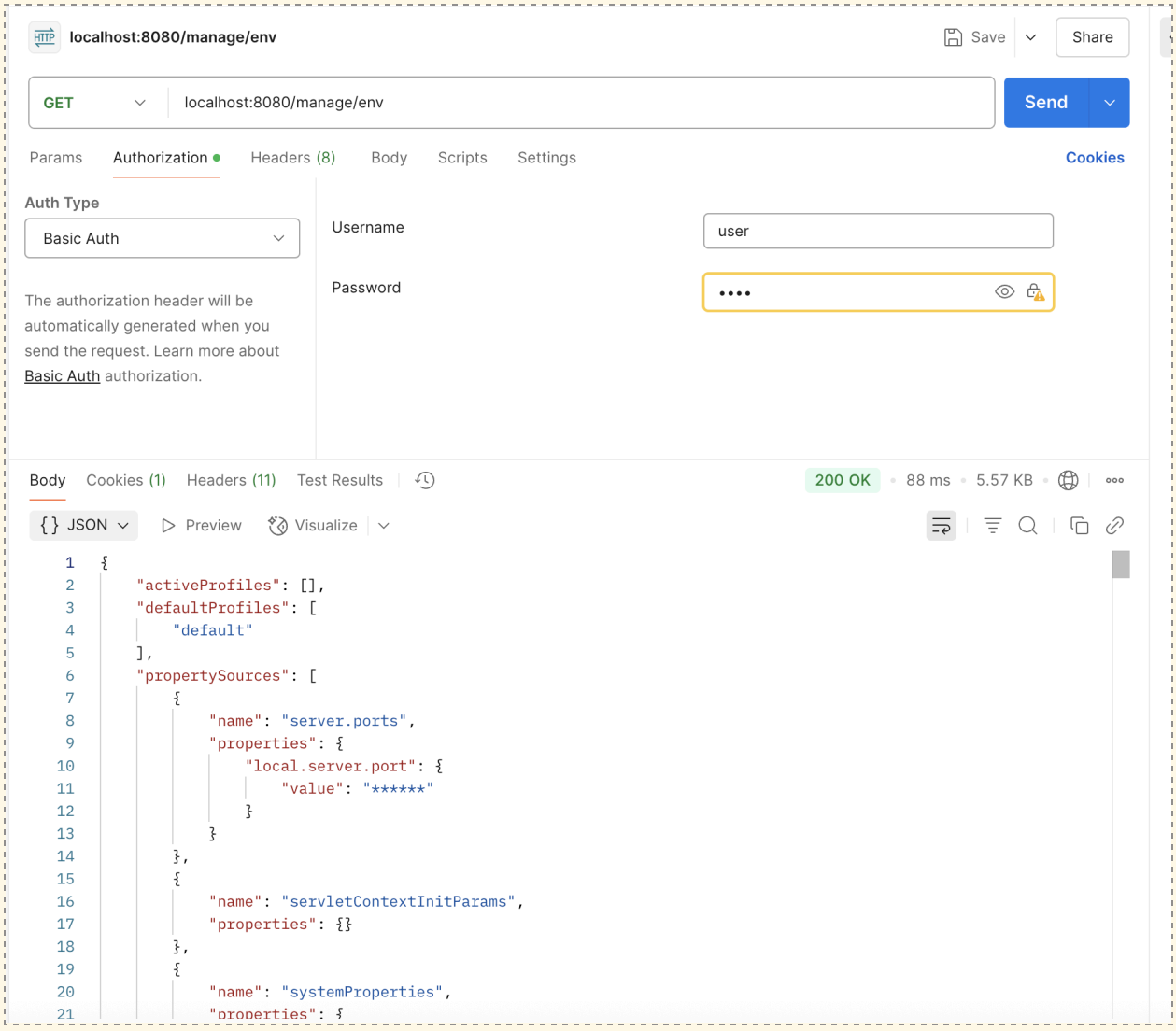
application.properties

```
spring.security.user.name=user
spring.security.user.password=pass
spring.security.user.roles=ADMIN
```

Trying to access /evn without providing the authentication, request denied:



Able to access, after successful Authentication:



Custom Actuator Endpoint:

Class annotated with @Endpoint(id = "custom endpoint name")

```
@Component
@Endpoint(id = "my-custom-stats")
public class MyCustomStatsEndpoint { ... }
```

Type of operation, our custom endpoint going to support:

@ReadOperation
(equivalent to HTTP GET)

@WriteOperation
(equivalent to HTTP POST)

@DeleteOperation
(equivalent to HTTP DELETE)

```
Component
@Endpoint(id = "my-custom-stats")
public class MyCustomStatsEndpoint {
```

```
    @ReadOperation
      public String readAll() {
      return "Hello, Spring Boot!";
      }

   @ReadOperation
      public String read(@Selector String name,
@Selector String message) {
      return "Hello: " + name + " msg for you is: " +
message;
      }
}
```

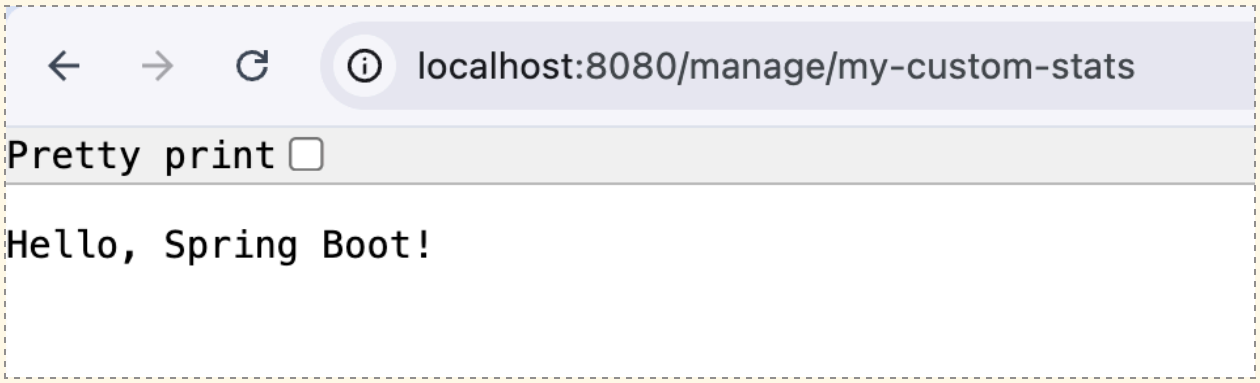
Return type can be any serializable object like Map, List, String, POJO, primitive etc.

/my-custom-stats/{name}/{message}
selector follows sequence

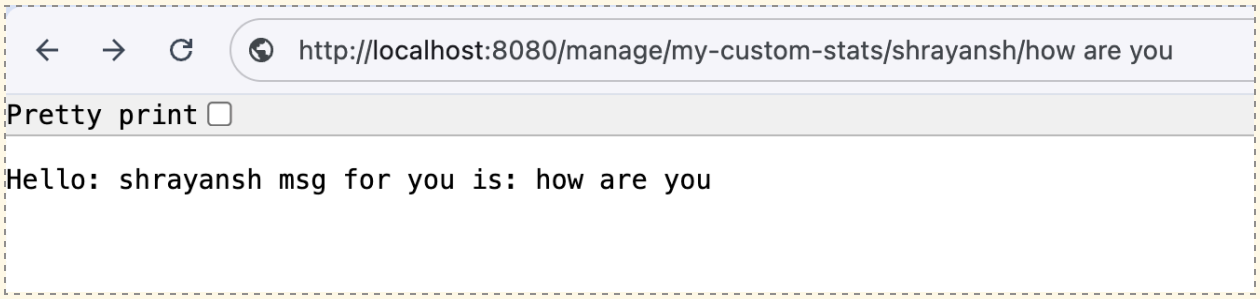
application.properties

```
spring.application.name=order-service

# by-default path is /actuator
management.endpoints.web.base-path=/manage

# Expose all actuator endpoints, by-default '/actuator/health' & '/actuator/info' endpoint is exposed
# '*' expose all the endpoints
# use comma separated like: health, info, metrics, loggers etc. to expose selected endpoints
management.endpoints.web.exposure.include=my-custom-stats,health,info

```

GET:



GET:

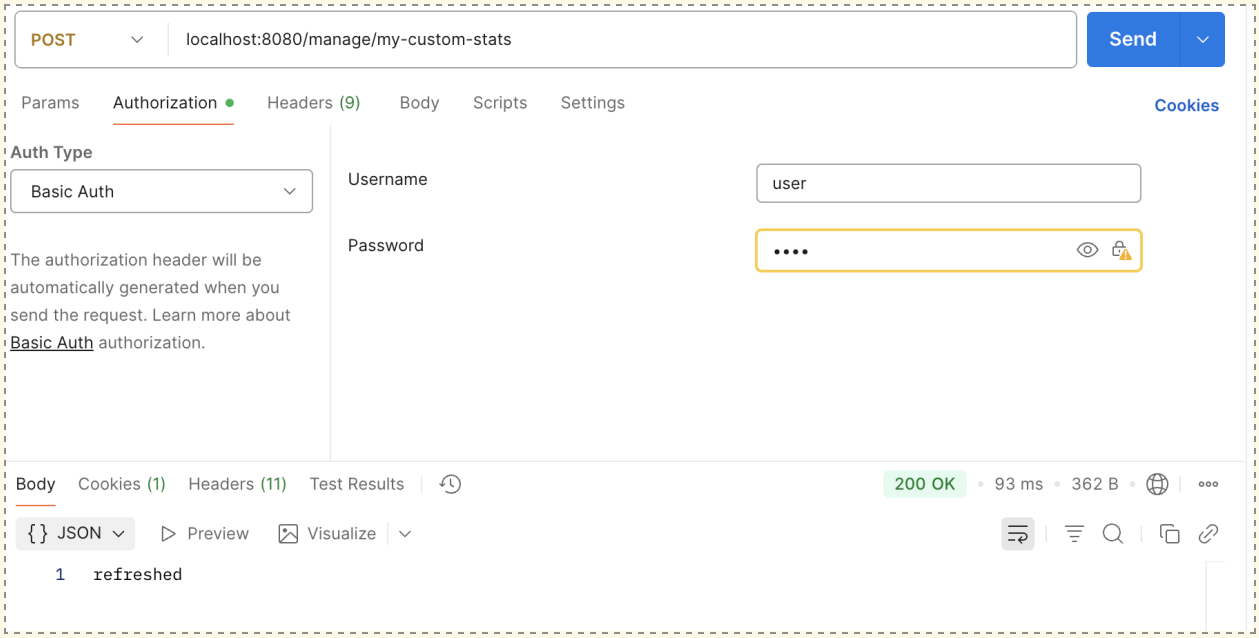


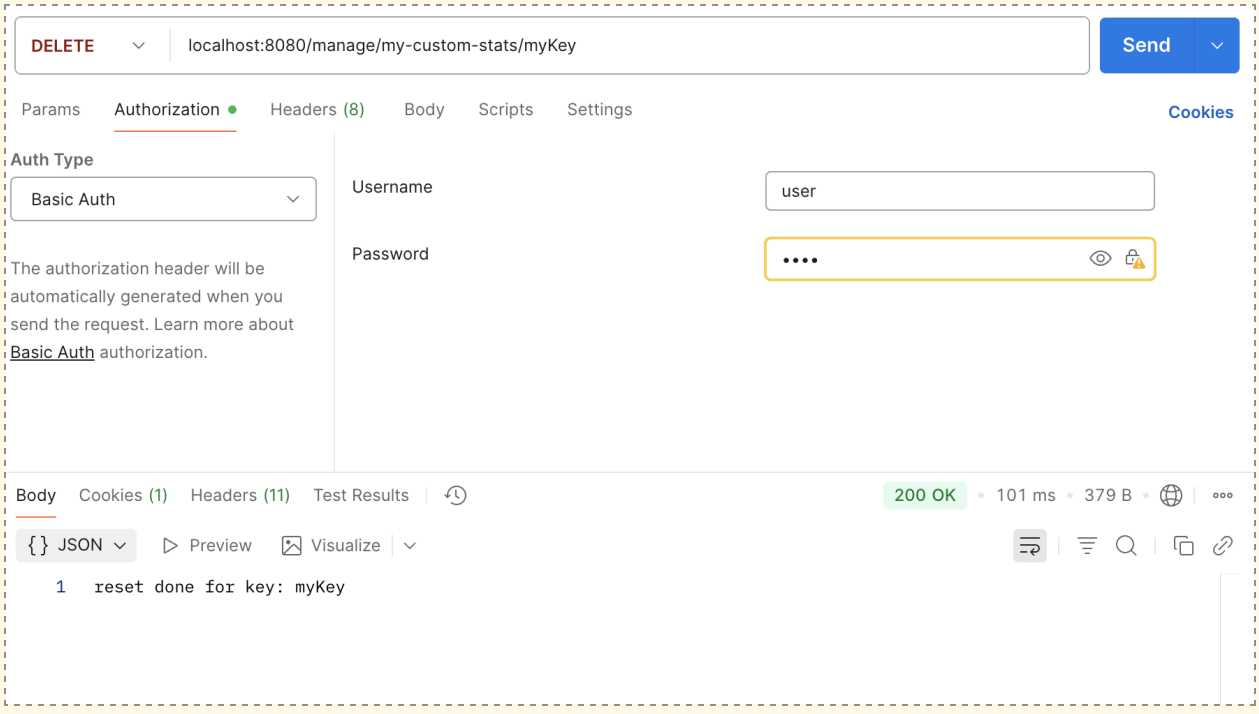
For POST and DELETE operation, it requires authentication

```
@Component
@Endpoint(id = "my-custom-stats")
public class MyCustomStatsEndpoint {

    @WriteOperation
```

```
    public String refresh() {
    // simulate say cache refresh
        return "refreshed";
    }

  @DeleteOperation
    public String remove(@Selector String key) {
    return "reset done for key: " + key;
    }
}
```

application.properties

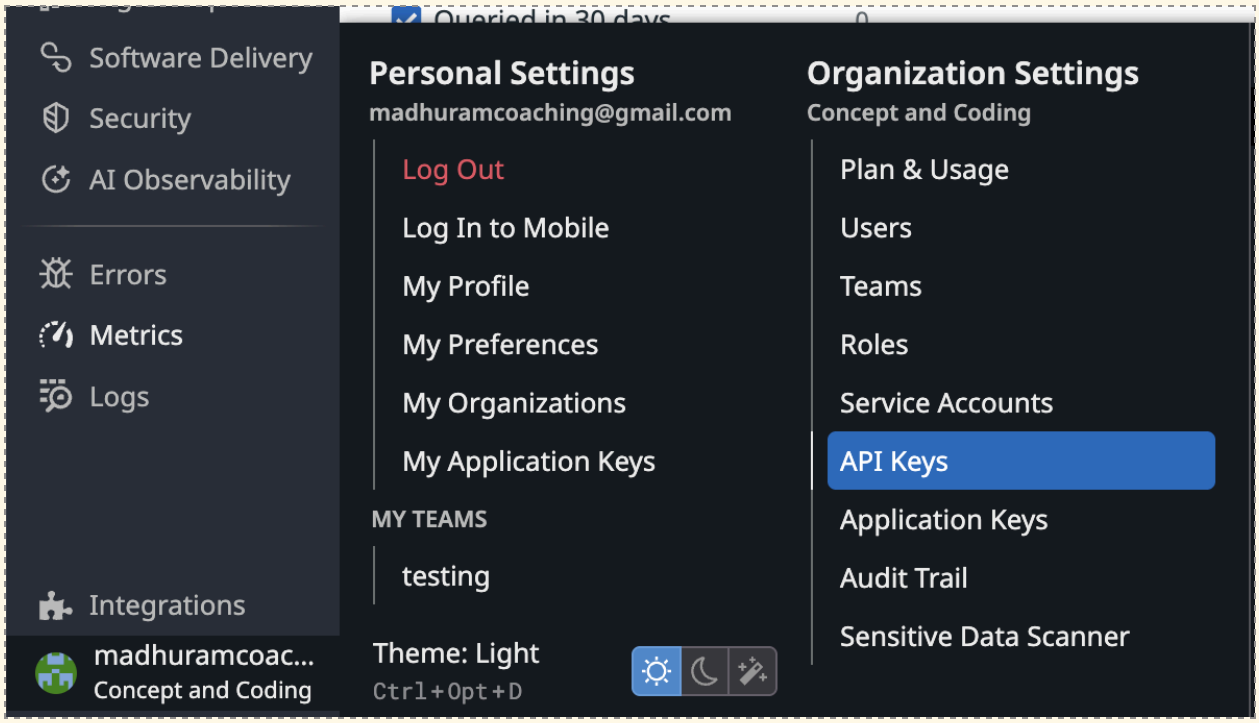




Pushing the metrics to Datadog (Monitoring platform)

we can also push to different other platforms like: Prometheus, CloudWatch etc..

Datadog (we can test with Free trial)



application.properties

```
management.datadog.metrics.export.apiKey=e59af51cb6ae3a24129da7230e5dd9f7
management.datadog.metrics.export.enabled=true
management.datadog.metrics.export.step=5s
```

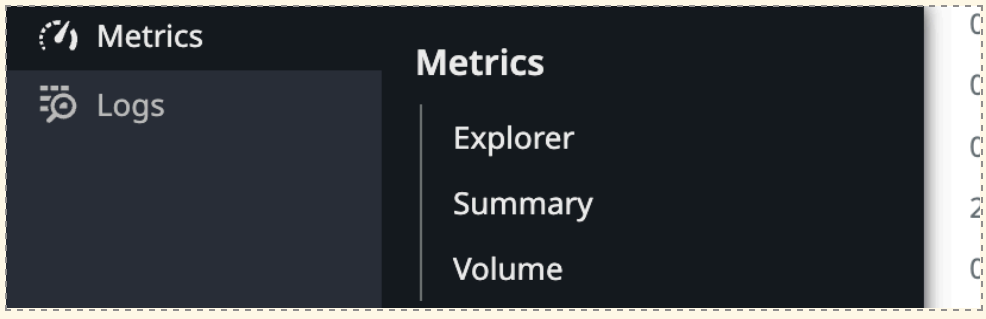
When its true then only it try to push the metrics
Every 5s it will push the metrics to datadog

POm.xml

```
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-registry-datadog</artifactId>
</dependency>
```

All the metrics of Spring Boot Actuator are automatically pushed to Datadog.

choose the metric we want to monitor
for example: *http.server.requests.count*



choose the metric we want to monitor

for example: *http.server.requests.count*